

Hybrid Search & Retrieval POC

https://github.com/aiyshwary/Search_Engine

Python

FAISS

BM25

OpenSearch

Sentence Transformers

Tesseract

OVERVIEW

A production-style multilingual hybrid search engine for Indian-language documents, combining BM25 lexical search and FAISS semantic retrieval to rank documents with snippet-level evidence across Tamil, Hindi, Telugu and more.

IMPACT

Built a multilingual hybrid search pipeline supporting Indian-language PDFs and DOCX files, combining BM25 (OpenSearch) and FAISS IVF-PQ for accurate document retrieval with chunk-level evidence and best-snippet selection.

TECHNICAL WALKTHROUGH

Elevator pitch

This repo is a small production-style hybrid search pipeline for PDFs/ DOCX: it ingests files, extracts text (OCR fallback), deduplicates, token-chunks documents, precomputes multilingual embeddings, builds FAISS ANN indexes, and serves a hybrid search that combines BM25 (OpenSearch) + ANN (FAISS) to rank whole documents with snippet-level evidence.

One-minute explanation

“We built a hybrid retrieval pipeline where BM25 provides precise lexical matches and FAISS (IVF+PQ + optional HNSW hot tier) provides semantic recall. Documents are normalized, chunked to ~400–800 tokens, embedded with a multilingual MiniLM, then indexed. At query time we run BM25 and ANN in parallel, union chunk candidates, aggregate per-doc signals (max BM25, max vector), and return ranked documents with a snippet and page offset.”

System components — 30-second walk-through

Ingest (optimized_ingest.py): PDF/DOCX extraction, Tesseract OCR fallback, per-page OCR confidence, language detection (fastText + Lingua).

Dedup (dedupe_simhash.py): document-level Simhash dedupe to avoid redundant indexing.

Chunking (chunker.py): tokenizer-based chunking with overlap (target tokens ~512, overlap ~64).

Embeddings (embedder.py): batch embed with sentence- transformers; normalize embeddings and write .npy + meta.

ANN build (build_faiss.py): create IVF+PQ (and HNSW for hot subsets) from embeddings.

Search (search_service.py): OpenSearch BM25 + FAISS ANN in parallel, union & aggregate to score docs.

KEY DETAILS

1. Ingests PDF and DOCX files with digital text extraction and Tesseract OCR fallback for scanned documents.
2. Detects document language using FastText (LID176) across Tamil, Hindi, Telugu, Kannada, Bengali and more.

3. Applies SimHash near-duplicate detection and token-aware chunking with sentence-based splitting.

4. Generates multilingual embeddings using Sentence Transformers and indexes them in FAISS IVF-PQ.

5. Hybrid BM25 and ANN retrieval with document-level score aggregation and best-snippet selection per result.

6. Embeds with paraphrase-multilingual-MiniLM-L12-v2 (L2-normalized, batch 256) and queries FAISS at nprobe=16; final document score is $0.5 \times \max(\text{BM25}) + 0.5 \times \max(\text{ANN})$ with reciprocal-rank ANN scoring.

7. Falls back to Lingua library when FastText confidence drops below 0.60, and normalizes ISO-639-1 codes to ISO-639-3 across the entire pipeline for consistent language filtering.

8. Parallelizes ingestion via ProcessPoolExecutor with $(\text{cpu_count} - 2)$ workers and runs BM25 and FAISS searches concurrently in separate threads before union.